Independent Academic Path Optimizer

Deployment Manual

AI-Assisted Class Scheduling Application

Document Type	Deployment Manual
Project	Independent Academic Path Optimizer
Course Deliverable	CS691 Sprint 2 Final Application
Version	v1.0.0
App URL	https://2026-paceu-capstone.github.io/intelligent-academic-path-optimizer
Prepared Date	August 2026

Revision History

Version	Date	Description
v1.0.0	August 2026	Initial user manual for Sprint 2 Wiki Submission

Deployment Architecture:

Table of Contents

1. Introduction

This manual is intended for developers who wish to deploy the project on their own local devices. It will highlight the required packages that are needed to be installed for the front and backend, the policies and setup required for AWS, and how to deploy all required technologies.

2. Frontend

a. PreRequistes:
   i. Node.JS - This is required to create and view React projects and install packages for the application.
   ii. EmailJS - An account is required for the contact page

b. Setup:
   i. Download/Clone the repository to your computer
   ii. Open up a CMD terminal or Powershell in the root folder that you downloaded, and follow the commands below. This installs the required node_modules that are listed in package.json

```
cd Frontend
npm install
```

c. Running:
    i. To run the application on your local computer, run the following command in your terminal/powershell. To stop running the program locally, press Ctrl-C and it will terminate the process.

```
npm run dev
```

d. Building (For third party Web Hosts)
    i. For building the site so it can be deployed, first terminate the local process (if it's running) and run the following command. This will create a folder called dist that contains the files for deploying your site to Website hosts such as Netlify, Wordpress or SquareSpace.

```
npm run build
```

e. Building (For Github Pages)
    i. In the Frontend folder, create an empty folder called ".github". In that folder create a new empty folder called "workflows".
    ii. Within the "workflows folder, create a new file called "deploy.yml" with the following lines inside it.

```
name: Deploy React app to GitHub Pages

on:
  push:
    branches:
      - main

permissions:
  contents: read
  pages: write
  id-token: write

concurrency:
  group: pages
  cancel-in-progress: true

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup Node
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: npm

      - name: Install dependencies
        run: npm ci

      - name: Build
        run: npm run build
```

```
    - name: Setup Pages
      uses: actions/configure-pages@v5

    - name: Upload artifact
      uses: actions/upload-pages-artifact@v3
      with:
        path: ./dist

deploy:
  environment:
    name: github-pages
    url: ${{ steps.deployment.outputs.page_url }}

  runs-on: ubuntu-latest
  needs: build

  steps:
    - name: Deploy to GitHub Pages
      id: deployment
      uses: actions/deploy-pages@v4
```

iii. After creating the file, create your own repository that contains just the Frontend folder and all of its contents (.github, public, src, index.html, etc.)

iv. Once your repository has all the Frontend files, go to Settings -> Pages and set the source under “Build and Deployment” to GitHub Actions. Once you do this, GitHub will automatically start the deployment of your site, and will provide you a link once it’s live.

f. Important Notes:

i. For EmailJS, after creating your service, template, and public key, replace the information on line 67 of the Contact.jsx file.

ii. When communicating with the backend, make sure to fill out the API_KEY and BASE_URL fields under /src/comp/callRequests.js. For more information, follow the instructions for setting up the Backend.

3. Backend

a. PreRequisites:

i. Python 3.12 - This version of Python is required for running FastAPI

ii. AWS - Several Access Keys and table names are required when storing or viewing course/user information

iii. Google Gemini - An API key is required to utilize the AI

b. Setup

i. Download/Clone the repository to your computer

ii. Open up a CMD terminal or Powershell in the root folder that you downloaded, and follow the commands below. This installs all the necessary packages for the backend

```
cd Backend
pip install "uvicorn[standard]" fastapi google-genai pypdf boto3
```

iii. After the packages are installed, the next thing that is required is to fill out the config.py file with all the required API keys and table names. The description for each variable is listed below.

```
BACKEND_API_KEY - This is the key that every request must have for them to have access to the backend. *Make sure the Frontend has this key so that it can communicate with the Web Server

AWS_ACCESS_KEY_ID - The access key that comes with your AWS developer account
AWS_SECRET_ACCESS_KEY - The secret key that comes with your AWS developer account
AWS_REGION1 - The main region that contains your tables or s3 buckets
AWS_REGION2 - If you have a secondary region that contains your tables or s3 buckets

S3_BUCKET - The S3 bucket name
DYNAMODB_TABLE - The main table name for all user information
SESSION_TABLE - The main table name for all user login sessions
INFO_TABLE - The name for the table that gets current available majors and semesters

GEMINI_API_KEY - The API key for Gemini

SESSION_LENGTH_DAYS - How long user login sessions should last before they expire.
```

c. Running the server (Locally)

i. To deploy the server locally, in the CMD terminal, run the following command. This will start the FastAPI server at the following link: http://127.0.0.1:8000

```
uvicorn main:app --reload
```

d. Important Notes:

i. For hosting on PythonAnywhere, keep in mind that the site does not natively support FastAPI frameworks and requires a manual setup. To create the webserver, follow this guide: https://help.pythonanywhere.com/pages/ASGICommandLine/#fastapi Once the environment is created, follow the steps under **b** to finalize installation.

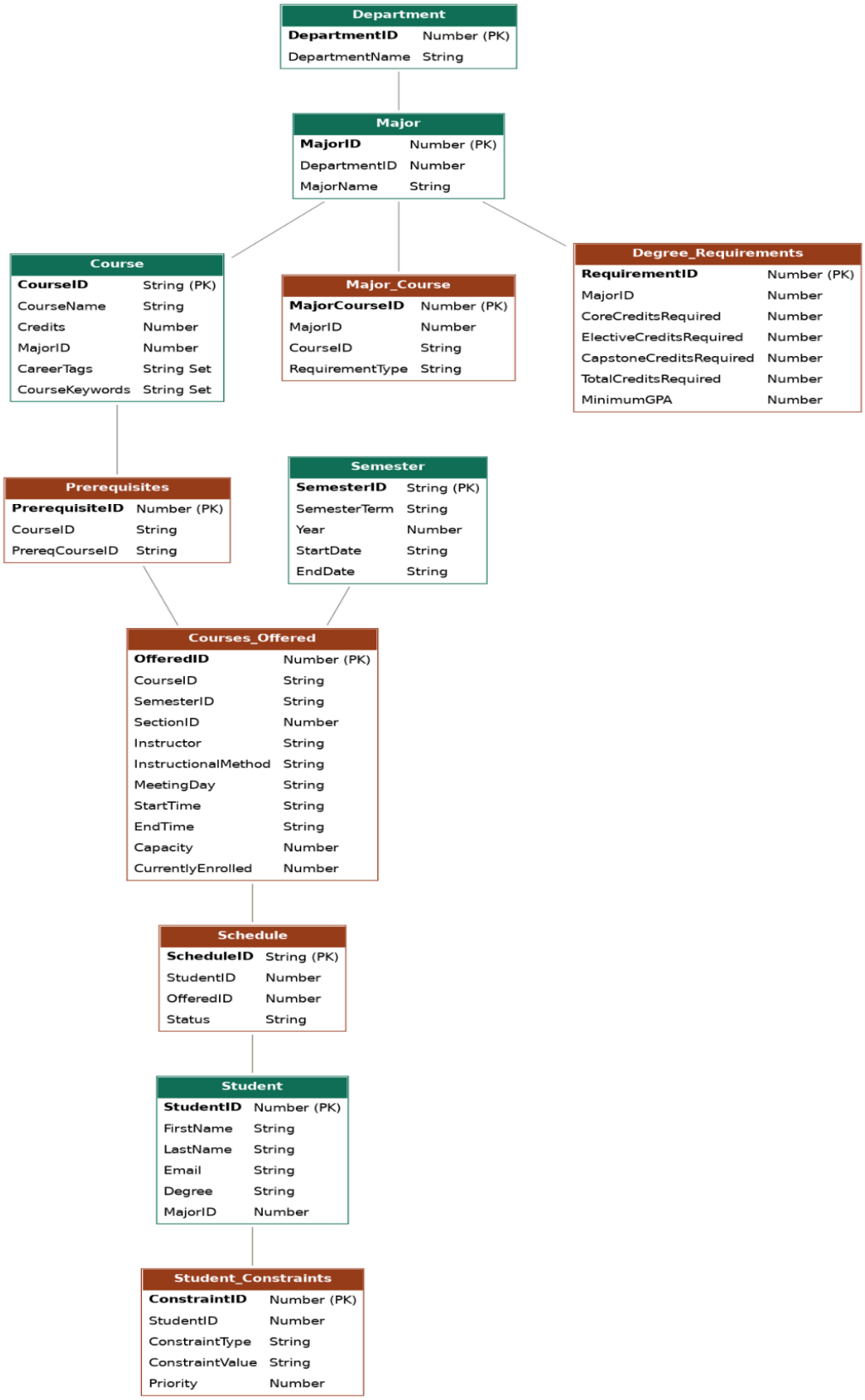
ii. To ensure that the FrontEnd can properly communicate with the Backend, make sure to set the BASE_URL variable in Frontend/src/comp/callRequests.js. The URL will either be the one listed

under **c** if you are running the server locally, or the https://<username>.pythonanywhere.com if you are running the server in PythonAnywhere.

4. Database

IAPO uses Amazon DynamoDB as its NoSQL database.

a. Database DB Data Mode:
    i. Originally designed relationally, translated into a NoSQL structure for DynamoDB
    ii. Tables:
        1. Foundation for academic data: Department, Major, Semester, Degree Requirements
        2. Student and Course data: Student, Course
        3. Relationship data: Major Course, Prerequisites
        4. Optimization layer: Courses Offered, Student Constraints, Schedules


Department
DepartmentID Number (PK)
DepartmentName String
Major
MajorID Number (PK)
DepartmentID Number
MajorName String
Course
CourseID String (PK)
CourseName String
Credits Number
MajorID Number
CareerTags String Set
CourseKeywords String Set
Major_Course
MajorCourseID Number (PK)
MajorID Number
CourseID String
RequirementType String
Degree_Requirements
RequirementID Number (PK)
MajorID Number
CoreCreditsRequired Number
ElectiveCreditsRequired Number
CapstoneCreditsRequired Number
TotalCreditsRequired Number
MinimumGPA Number
Prerequisites
PrerequisiteID Number (PK)
CourseID String
PrereqCourseID String
Semester
SemesterID String (PK)
SemesterTerm String
Year Number
StartDate String
EndDate String
Courses_Offered
OfferedID Number (PK)
CourseID String
SemesterID String
SectionID Number
Instructor String
InstructionalMethod String
MeetingDay String
StartTime String
EndTime String
Capacity Number
CurrentlyEnrolled Number
Schedule
ScheduleID String (PK)
StudentID Number
OfferedID Number
Status String
Student
StudentID Number (PK)
FirstName String
LastName String
Email String
Degree String
MajorID Number
Student_Constraints
ConstraintID Number (PK)
StudentID Number
ConstraintType String
ConstraintValue String
Priority Number

b. Tables Creation:
    i. Manually through the AWS Management Console
    ii. Region
    iii. Creation Includes: Table Name, Partition Key
    iv. JSON attribute formats used

c. Load Data:
    i. Data originally existed in a SQL-based design
    ii. Populated manually into DynamoDB
    iii. Create Table (explained above)
        ↓
        Open DynamoDB Table
        ↓
        Create Item
        ↓
        Enter Attributes/Values
        ↓
        Save Item
        ↓
        Verify Item

d. Backend Integration:
    i. Backend built with Python and FastAPI, hosted on PythonAnywhere.
    ii. Acts as middleman between frontend, AWS database, and Gemini API.
    iii. Frontend never directly accesses AWS data.
    iv. Has a helper file that handles the AWS DynamoDB connection.
    v. Requests for course, prerequisite, or user data are routed through backend's flexible structure to the appropriate handler, which updates DynamoDB.

e. Verify DynamoDB:
    i. Open each table and confirm each item present as expected.
    ii. Check JSON formats (S, N, SS) output in each item for accuracy.
    iii. Backend confirms data match through testing with what's shown on application.

f. Troubleshooting:

Problem	Possible Cause	Resolution
Data appeared under wrong data type	Incorrect attribute format used during manual entry	Reformat using the correct attribute . Ex: String to Numeral (“S”:... > ”N”: ….)
Backend fail to connect to correct Dynamodb table	Region or table name incorrect between backend and AWS	Confirm region and Table name in AWS match with Backend config
Item fails to save	Missing/repeated partition key	Confirm every item partition key and attribute type before creating + saving item.

g. Recovery:
   i. Manually reimport CSV files (backup) if data gets lost/corrupted.
   ii. If CSV files are unavailable, use original SQL data.